Intelligent E-Commerce Search with Retrieval Augmented Generation

Sahana Srinivasa Babu - 019175975
Bishakha - 019178354
Kevin Anto Jesus William Basker - 017478604
Shiyuan Wang - 015632175

***Abstract*—Traditional keyword-based e-commerce search systems often fail to capture user intent, returning irrelevant results when product descriptions do not contain exact query terms. This project presents an intelligent e-commerce search engine based on Retrieval-Augmented Generation (RAG) that improves product discovery through semantic understanding of user queries. A multi-category e-commerce dataset is indexed using dense vector embeddings stored in a ChromaDB vector database. At query time, semantically relevant products are retrieved and provided as context to Large Language Models (LLaMA), to generate accurate and context-aware responses. Retrieval quality is further enhanced through category-based metadata filtering, hybrid retrieval combining BM25 and SBERT, reranking using BGE, and Redis-backed semantic caching for reduced latency. The system has been evaluated using Precision@5, NDCG@5, response latency, faithfulness, and answer relevancy across multiple retrieval configurations. Experimental results show that the Hybrid Retrieval pipeline (BM25 + SBERT + BGE) achieved the best overall performance, obtaining the highest Answer Relevancy score of 0.7516, Precision score of 0.96, NDCG score of 0.9839, and Faithfulness score of 0.811, demonstrating superior retrieval quality and factual grounding. The results demonstrate that combining lexical and semantic retrieval methods with reranking and caching significantly improves both search relevance and system efficiency, making the proposed solution suitable for real-time e-commerce search applications.**

Project Repository: GitHub Repository

I. Introduction

E-commerce platforms fully depend on search systems to help users discover relevant products. But the most traditional search engines are based on keyword matching techniques, which often fail to capture the true intent behind user queries. For example, consider the query "budget phone with good battery life." A keyword-based system may return products that explicitly contain terms like "budget" or "battery" in their descriptions, missing relevant items described differently (eg. "long-lasting battery" or "value for money smartphone"). On the other hand, a Retrieval-Augmented Generation (RAG) system interprets the semantic meaning of the query, retrieves products with similar intent, and generates a response such like "Here are some affordable smartphones known for strong battery performance, including models with 5000mAh batteries and high user ratings." This highlights how RAG based approaches can provide more context aware, flexible, and user-friendly results compared to the strict keyword matching systems.

In this project, we are implementing an intelligent e-commerce search system using a LangChain-based RAG pipeline. Product data is first preprocessed and transformed into vector embeddings, and these embeddings are stored in ChromaDB. At query time, user queries are embedded into the same vector space and relevant products are retrieved using cosine similarity-based vector search. The retrieved results are then provided as context to a Large Language Model (LLM) which generates context-aware responses for the user.

We also plan to use additional retrieval strategies such as BM25 (keyword-based search), SBERT, and hybrid approaches. To further improve the performance, we are also planning to implement re-ranking and caching for latency reduction.

The system is evaluated across multiple configurations Precision@K, Response Latency, Faithfulness and Answer Relevancy.

II. Background/Related Work

A. Keyword-Based E-Commerce Product Search

Keyword-based search engines have traditionally been the foundation of product discovery in e-commerce platforms. These systems retrieve products by matching user query terms with textual information contained in product titles, descriptions, categories, and metadata. Information retrieval techniques such as the Vector Space Model (VSM), Term Frequency–Inverse Document Frequency (TF-IDF), and probabilistic ranking methods have been widely employed to rank products according to their relevance to a user's query [1], [2]. BM25 has demonstrated strong performance in large-scale retrieval systems and remains a common baseline for e-commerce search applications [3].

B. Limitations of Keyword-Based Search

Despite their widespread adoption, keyword-based search systems face several challenges in understanding user intent. Their dependence on exact or near-exact term matching often leads to poor retrieval performance when users employ synonyms, abbreviations, misspellings, or natural language expressions that do not explicitly appear in product descriptions. For example, a query such as "comfortable running shoes" may fail to retrieve relevant products if product listings contain terms like "athletic sneakers" instead of "running shoes."

Additionally, keyword-based systems struggle with ambiguous queries, long-tail search terms, and conversational search requests, all of which have become increasingly common in modern e-commerce environments [6].

C. Towards Semantic Search

To address the shortcomings of lexical retrieval, researchers have explored semantic search approaches that leverage distributed word representations and deep learning techniques. Word embedding models such as Word2Vec and GloVe introduced the ability to represent words in continuous vector spaces, enabling the capture of semantic similarities between terms [7], [8]. More recently, transformer-based language models, including BERT and Sentence-BERT (SBERT), have significantly improved semantic understanding by generating contextual embeddings for sentences and documents [9], [10]. Dense retrieval methods encode both queries and product descriptions into vector embeddings and retrieve products based on similarity measures such as cosine similarity.

D. Related Work

Recent research has focused on transformer-based retrieval and reranking methods. Nogueira and Cho demonstrated that BERT-based rerankers substantially improve retrieval effectiveness by modeling deeper interactions between queries and documents [14]. Sentence-BERT further enables efficient semantic retrieval by producing fixed-length sentence embeddings suitable for large-scale vector search applications [10]. In addition, Retrieval-Augmented Generation (RAG) frameworks combine retrieval systems with large language models to generate context-aware responses while grounding outputs in retrieved documents, making them increasingly relevant for intelligent e-commerce search systems [11].

III. Dataset and Preprocessing

A. Description of the dataset

The E-Commerce Products Dataset used in this project is sourced from Kaggle and contains detailed product information across multiple categories. This diverse set of features makes the dataset suitable for building and evaluating search and recommendation systems.

Dataset Source: Kaggle E-Commerce Products Dataset

B. Product Catalog Description

All products share the same fields as described in Table 1. For all products, the number of fields n is fixed to 6 while the number of instances k varies between fields. The dataset contains 6,506 products across 28 categories sourced from Amazon. The last field, Search Text, is a structured concatenation of product attributes used as the retrieval document passed to the vector store.

Fields	Instances
Title	Product name (single value per product)
Category	Main product category (e.g., Phones, Cameras)
Numeric	Rating score (0–5), number of ratings, price in USD
Image	Product image URL from Amazon CDN
Source	Source CSV file (subcategory label, 28 categories)
Search Text	Concatenated retrieval string: name + category + price + rating + review count

Six fields selected for representing products.

C. Data cleaning

Preprocessing dataset was implemented to ensure data consistency, reliability, and compatibility with downstream retrieval and embedding models. The preprocessing stage consisted of the following operations:

- **Dataset Integration:** The original Kaggle dataset consisted of 28 category-specific CSV files which were merged into a unified product catalog.
- **Missing Value and Duplicate Handling:** No missing values were identified in critical attributes. A total of 12 duplicate entries introduced during file concatenation were removed.
- **Schema and Category Normalization:** Column names and category labels were standardized across all source files to maintain schema consistency.
- **Numeric Feature Cleaning:** The fields `actual_price` and `no_of_ratings` were converted from floating-point representations to integer values. Product ratings were validated to ensure they remained within the range of 0–5, while price values were verified to be positive and non-zero.
- **Unicode and Text Normalization:** Product names were normalized using Unicode NFKD decomposition to remove encoding artifacts and improve textual consistency for embedding generation.
- **Search Corpus Construction:** A unified retrieval field, denoted as `search_text`, was generated by concatenating key product attributes including name, category, price, rating, and review count into a single semantic representation.

D. Exploratory Data Analysis

Prior to building the retrieval pipeline, an exploratory analysis was conducted on the preprocessed dataset to validate data quality and justify key design decisions in the RAG system.

1) Category Distribution: The dataset spans 28 product categories with most categories containing 250 products each, as shown in Figure 1. A small number of categories such as Mens Watches (14 products) and Phones (170 products) contained fewer entries due to limited availability in the source data. This distribution confirms the semantic diversity of the catalog, highlighting the need for a retrieval approach based on meaning rather than keyword or category filtering alone.

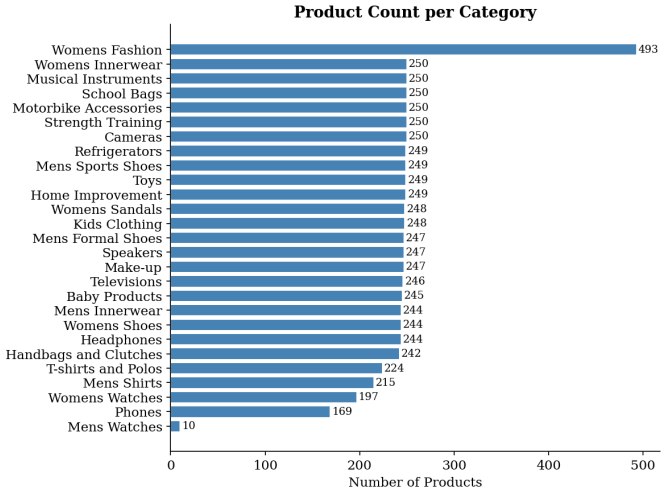

Fig. 1: Product count per category across the 28-category dataset.

2) Search Text Length Distribution: The length of the constructed `search_text` field was analyzed to determine whether document chunking was necessary prior to embedding. As shown in Figure 2, the average search text length was 245 characters with a maximum of 346 characters — well within the 512-token input limit of the `all-MiniLM-L6-v2` embedding model. This confirmed that each product record could be embedded as a single unit without truncation or chunking.

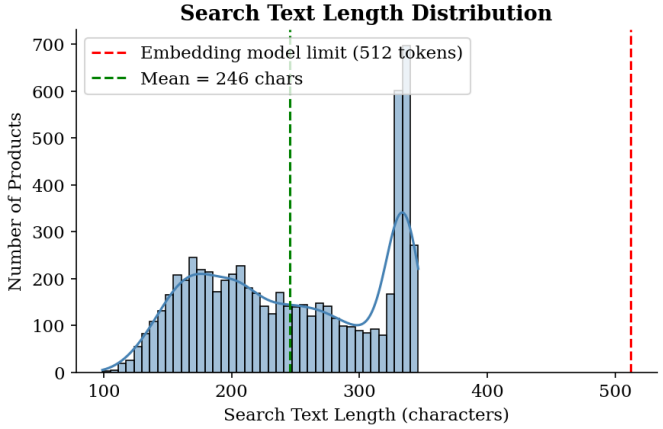

Fig. 2: Distribution of search text lengths. All records fall below the 512-token embedding model limit, eliminating the need for chunking.

3) Average Rating vs. Average Price per Category: Figure 3 presents a bubble chart comparing the average price and average rating across all 28 categories, where bubble size represents the number of products in each category. High-value electronics such as Televisions and Refrigerators occupy the high-price region, while apparel and accessories cluster in the low-price region. Despite this price variation, average ratings remain consistently between 3.9 and 4.3 across all categories. This uniformity in ratings indicates that rating alone is insufficient as a retrieval signal, further motivating the use of semantic search.

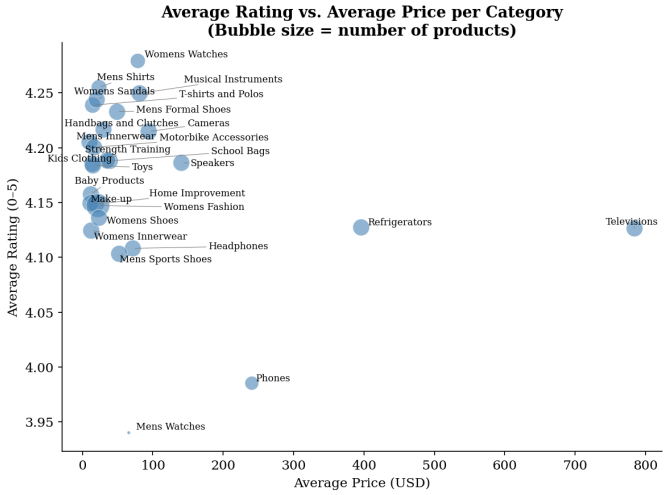

Fig. 3: Average rating vs. average price per category. Bubble size reflects product count.

E. Data chunking and embedding

Chunking is a technique used to split large text documents into smaller segments so that they can be processed by embedding models with input size limitations. Product information was relatively short. Each record was averaging around 245 characters and a maximum of 346 characters. Since these lengths are well within the limits of the embedding model, chunking is not required.

F. Vector database ingestion

Vector database ingestion is the process of storing the generated embeddings along with their corresponding metadata into a vector database to enable efficient semantic search and retrieval. In this project, ChromaDB was used as the vector database for storing product embeddings. The HuggingFaceEmbeddings object converts each document into a dense vector representation that captures its semantic meaning. After generating embeddings for each product record, the data was ingested into the database in a structured format. Each entry in the database consists of:

- A unique product ID
- The embedding vector representing the product
- The search text (combined product information)
- Additional metadata such as category, price, ratings, and number of reviews

Before ingestion, embeddings were validated. Metadata fields were also verified to maintain uniform structure. This enables filtering during search queries like by category or price range in addition to semantic similarity. Once ingested, ChromaDB automatically indexes the embeddings. It allows fast similarity

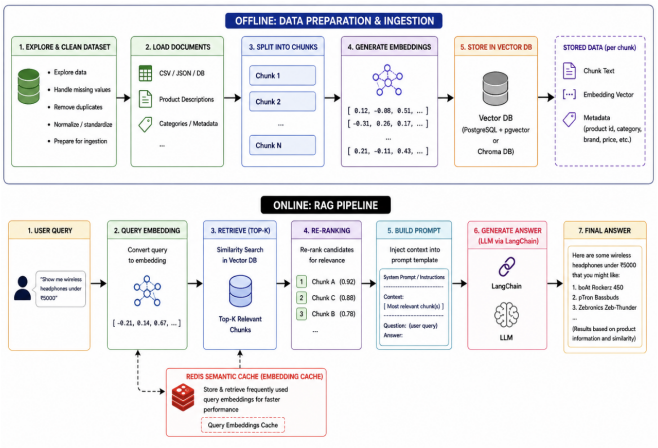

Fig. 4: RAG Pipeline

search using cosine distance. This enables the system to retrieve the most relevant products based on semantic similarity, rather than relying solely on exact keyword matches.

IV. Technical Approach

A. RAG Pipeline

The main idea in the project is to use Retrieval-Augmented Generation (RAG) to improve the accuracy, relevance, and reliability of Large Language Model (LLM) responses by combining information retrieval with text generation. Instead of relying solely on the model's pre-trained knowledge, the system retrieves relevant documents from a knowledge base (ChromaDB Persistent Storage) and provides them as context to the LLM before generating an answer. Key Steps involved in implementing RAG Pipeline are as follows :

- Document Ingestion and Chunking
- Embedding generation and Vector Storage
- Retrievers
- Reranking
- Answer generation with LLMs.

In the Retrieval-Augmented Generation (RAG) pipeline, Document Ingestion and Chunking , Embedding generation and Vector Storage are all done as first step with ChromaDB. The vector database enables efficient semantic search by retrieving documents that are most similar to a user's query.The most relevant documents are then retrieved and provided as context to a Large Language Model (LLM), improving the accuracy and relevance of generated responses.

Reranker - BGE

BGE Reranker is a neural ranking model used to improve the relevance of retrieved documents in Retrieval-Augmented Generation (RAG) systems and search applications. A reranker performs a second-stage evaluation to determine which documents are most relevant to the user's query. It works by taking a query-document pair as input and processing them together through a transformer-based cross-encoder architecture. The model analyzes the semantic relationship between the query and each candidate document, generating a relevance score for every pair. Documents are then reordered based on these scores, with the highest-ranked results passed to downstream components such as large language models.

B. Retrieval Layer

BM25

To ensure that exact keyword matches (such as specific brand names or product types) are not missed, we first implemented a BM25Okapi baseline. This track is fast and highly effective for exact queries, but it struggles with vocabulary mismatch.

- Preprocessing: We tokenized the product corpus by converting the text to lowercase and splitting it into individual words.
- Process: When a user inputs a query, it is split in the same way. The BM25 algorithm scores each document based on term frequency and document length normalization.

SBERT

To capture the actual intent behind user queries, we integrated a semantic search component using Sentence-BERT

- Embedding Generation: We utilized LangChain's vector store utilities. During the offline phase, all product descriptions were converted into dense vector embeddings.
- Query Transformation: At runtime, when a query like "i am planning to go to beach" is received, the system calls `embeddings.embed_query(query)` to transform the raw text into a query embedding.
- Vector Matching: The vector store then performs a similarity search (using cosine similarity) to find the top k documents closest to the query embedding in the vector space. This allows us to retrieve contextually relevant items even if they don't share exact words with the query.

Hybrid Search

We noticed that relying solely on keyword search often fails to capture context (e.g., searching for "beach" might not surface a "swimsuit"), while relying purely on semantic search can sometimes miss specific, exact keywords. To solve this, we combined BM25 and SBERT into a hybrid search system to capture both the precision of keyword matching and the depth of semantic understanding.

- Data Preparation and Index Tracking: Before performing any cross-model retrieval, we enriched the metadata of our existing document collection (docs) by injecting a unique reference index (idx) into each item.
- Retrieval: When a user query is received, it is passed to both retrievers simultaneously. BM25 fetches its top candidates based on keyword matching, while SBERT converts the query into a query embedding to fetch its top candidates based on vector proximity. The pipeline maps the sorted outputs of both BM25 and SBERT into rank lookup dictionaries (bm25_mapping and sbert_mapping).
- Hybrid Logic via Reciprocal Rank Fusion (RRF): To combine BM25 keyword search and SBERT semantic search, we use Reciprocal Rank Fusion (RRF) instead of

directly merging their scores, as the two methods produce scores on different scales. RRF ranks documents based on their positions in each retriever's results. The top 100 documents from both methods are combined into a candidate pool, with missing documents assigned a penalty rank of 1000. Using k=60, RRF scores are calculated and aggregated for each document. The documents are then sorted by their RRF scores, and the top 5 results are retrieved and displayed from the original document collection.

C. List of Experiments

1) **RAG with SBERT Retriever and BGE Reranker**
   This setup uses SBERT (Sentence-BERT) embeddings to perform semantic document retrieval. Retrieved documents are then re-ranked by a BGE Reranker to improve relevance before passing context to the LLM.
2) **RAG with SBERT Retriever, BGE Reranker, and Redis**
   This setup uses SBERT for semantic retrieval and BGE Reranker for relevance optimization with Redis Semantic cache store for caching layer for fast document indexing and retrieval. It helps in improving query latency, and production-readiness while maintaining retrieval accuracy.
3) **RAG with BM25 Retriever**
   This setup uses BM25, a traditional keyword-based retrieval algorithm, to find relevant documents . It relies on term frequency and inverse document frequency rather than embeddings. It is simpler and computationally efficient but may miss semantically related content.
4) **RAG with Hybrid Retriever and BGE Reranker**
   This setup combines BM25 keyword search and dense semantic retrieval (SBERT embeddings) in a hybrid retrieval strategy. It retrieves results from both methods and are merged and re-ranked using a BGE Reranker. It balances lexical matching and semantic understanding, often achieving the best retrieval performance.

In order to improve performance , we tried adding a few extra steps . One of them is category filtering and the other is Redis Semantic cache.

Category Filtering

Category filtering is used to narrow the search space before performing vector retrieval. Based on the user's query, the system identifies the most relevant product or document category and restricts retrieval to documents belonging only to that category.A metadata filter is applied and only documents matching the selected category are retrieved. The retrieved documents are then reranked (using BGE Reranker) and passed to the LLM for answer generation.This helps the reranker focus on more relevant Products , reducing irrelevant document retrieval.

Caching Results with Redis

Instead of repeating these steps for every query, we cache the final ranked results. This means that after we have performed the RRF and Rerank steps for a specific query, we store that finalized list in Redis. If the same query is entered again, the system skips the model inference and the ranking logic entirely and returns the cached results instantly.

Logic for Caching

Logic for Caching

- **Check the Cache:** When a user submits a product query, Redis checks whether a semantically similar query has been processed before. If found, the system can reuse previous retrieval results instead of performing retrieval again, reducing computation and response time.
- **Cache Hit:** A cache hit occurs when the similarity score exceeds the predefined threshold (**0.92**). The system retrieves the cached documents from Redis Context Cache, converts them into LangChain Document objects, and returns them directly.
- **Cache Miss:** If no matching context exists or the cached entry has expired, the system performs Chroma retrieval and BGE reranking. The retrieved documents are then stored in Redis with a **24-hour TTL** for future reuse.
- **LLM Generation:** The retrieved documents are passed to the LLM as context, which generates the final product recommendations returned to the user.

V. Experimental Result

A. *Precision@5 and NDCG@5*

To evaluate retrieval quality, Precision@5 and NDCG@5 were used. Precision@5 measures the proportion of relevant products among the top 5 retrieved results, while NDCG@5 evaluates both relevance and ranking order.

TABLE I: Precision@5 and NDCG@5 Across RAG Pipelines

Pipeline	Precision@5	NDCG@5
BM25	0.4333	0.6942
Vector(SBERT + BGE)	0.9867	0.9888
Hybrid + BGE	0.9600	0.9839

BM25 achieved the lowest Precision@5 score of 0.4333. This means that less than half of the products returned in the top 5 results were relevant on average. Since BM25 relies on keyword matching, it may have difficulty handling queries that use different wording from the product descriptions.

The Hybrid retriever achieved a Precision@5 score of 0.96 and an NDCG@5 score of 0.9839. Compared with BM25, the Precision@5 score increased by more than 50 percentage points. This indicates that combining keyword search and semantic search helped retrieve more relevant products for user queries.

The Vector retriever achieved the best overall performance, with a Precision@5 score of 0.9867 and an NDCG@5 score of 0.9888. These results suggest that semantic embeddings were

highly effective at understanding the meaning of user queries and retrieving products that matched user intent.
One interesting observation is that the difference between the Hybrid and Vector retrievers was much smaller than the difference between BM25 and the semantic retrieval methods. This suggests that semantic understanding played a major role in retrieval performance for this dataset.

B. *Answer Relevancy*

1) Methodology: Answer relevancy measures how semantically aligned each generated answer is with the original user query. Rather than requiring a reference answer, the metric computes the cosine similarity between the embedding of the query and the embedding of the generated answer using the `sentence-transformers/all-MiniLM-L6-v2` model. Scores range from 0 to 1, where higher values indicate that the answer directly and specifically addresses what was asked. Each pipeline was evaluated across all test queries from `test_queries.json`, with a 3-second inter-query delay to respect Groq API rate limits.

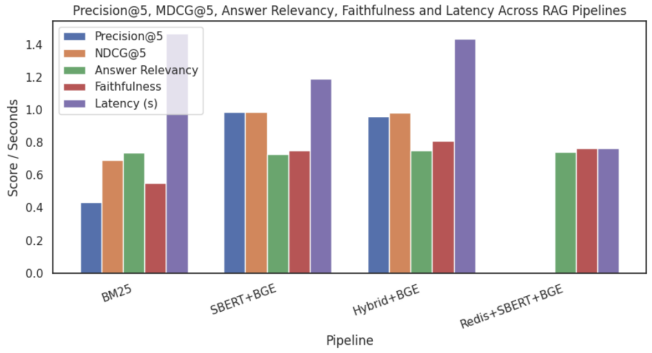

Fig. 5: Precision@5, NDCG@5, Answer Relevancy, Faithfulness and Latency Across RAG Pipelines

2) Results: Three retrieval pipelines were evaluated against two LLMs — **LLaMA-3.1-8B-Instant** and **Qwen-2.5-72B-Instruct** (served via the Groq API). The Redis-cached pipeline was excluded from this run as it requires a live Redis instance.

3) Analysis: **Hybrid retrieval achieves the highest answer relevancy overall.** With LLaMA, the Hybrid (BM25+Vector+BGE) pipeline scores 0.7516, outperforming BM25+BGE (0.7394) and Vector (0.7263). This confirms that combining keyword and semantic search via Reciprocal Rank Fusion (RRF) retrieves more contextually complete product information, enabling the LLM to generate answers that more precisely address the user's intent.

LLaMA-3.1-8B consistently benefits from richer retrieval. For LLaMA, scores improve monotonically with retrieval sophistication: Vector → BM25 → Hybrid (0.7263 → 0.7394 → 0.7516). This suggests the smaller model relies heavily on the quality and coverage of retrieved context to stay on-topic.

Qwen-2.5-72B favours semantic over keyword retrieval. Qwen scores highest on the Vector pipeline (0.7338) and drops significantly on BM25+BGE (0.7046) — the lowest score across all configurations.

TABLE II: Answer Relevancy Scores by Pipeline and Model

Pipeline	LLaMA-3.1-8B	Qwen-2.5-72B
Vector (SBERT+BGE)	0.7263	0.7338
BM25	0.7394	0.7046
Hybrid (BM25+Vector+BGE)	**0.7516**	0.7322

LLaMA outperforms Qwen on hybrid and BM25 pipelines. Despite being a much smaller model (8B vs. 72B parameters), LLaMA-3.1-8B-Instant achieves higher answer relevancy on both BM25+BGE and Hybrid pipelines. This may be attributed to Qwen's tendency to generate more elaborate, verbose responses that — while detailed — diverge semantically from the concise query embedding, penalising its cosine similarity score.

The best overall configuration is **Hybrid (BM25+Vector+BGE) with LLaMA-3.1-8B-Instant**, achieving an answer relevancy score of **0.7516**.

C. *Faithfulness Analysis*

The faithfulness evaluation shows that the quality of retrieved documents has a direct impact on how well the generated answers are supported by evidence. Among all the pipelines tested, the Hybrid Retrieval approach achieved the highest faithfulness score of **0.811**. By combining BM25 keyword search with SBERT-based semantic retrieval and BGE reranking, the system was able to retrieve more relevant information to support its responses. Faithfulness evaluation was conducted on an average on 45 queries.

For each claim extracted from the generated answer, cosine similarity was computed against all retrieved context chunks:

$$\text{sim}(c_i, d_j) = \frac{\mathbf{c_i} \cdot \mathbf{d_j}}{\|\mathbf{c_i}\|\|\mathbf{d_j}\|}$$

where c_i represents an answer claim and d_j represents a retrieved context chunk. A claim was considered supported if its maximum similarity score exceeded a predefined threshold of 0.40.

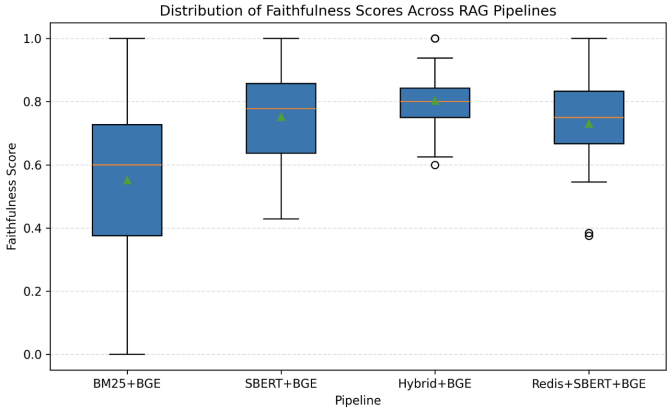

Fig. 6: Distribution of Faithfulness score across RAG Pipeline.

The faithfulness score was calculated as show from the formula below:

$$\text{Faithfulness} = \frac{\text{Number of Supported Claims}}{\text{Total Number of Claims}}$$

A higher faithfulness score indicates that a larger proportion of the generated answer is supported by the retrieved evidence. The Vector Retrieval pipeline achieved a faithfulness score of **0.752**, which was significantly higher than the BM25 baseline score of **0.552**. This indicates that semantic retrieval is more effective than keyword-based retrieval alone for finding relevant product information in an e-commerce setting.

The Redis-enhanced pipeline achieved a faithfulness score of **0.746**, which was similar to the Vector Retrieval approach. Since Redis is primarily used as a caching layer, its main benefit is reducing response time rather than improving retrieval quality. Therefore, its faithfulness performance was expected to be close to that of the underlying retrieval system.

The results show that combining keyword-based and semantic retrieval produces the most reliable responses, with answers better supported by the retrieved context.

D. Latency Analysis

Latency measures the time required for a RAG pipeline to generate a response after receiving a user query. Lower latency indicates faster response generation and a better user experience.

For each pipeline, the response time was measured for every query using Python's `time.perf_counter()` function. We used 10 queries with 3 repeated queries to evaluate latency performance on all pipelines. The average latency was then computed across all evaluation queries using:

$$\text{Average Latency} = \frac{1}{N}\sum_{i=1}^{N} t_i \tag{1}$$

where (N) represents the total number of evaluation queries and $(t_i) denotes the response time (in sec) for query (i)$.

TABLE III: Average Latency Across RAG Pipelines

Pipeline	Average Latency (s)
BM25	1.469
SBERT + BGE	1.192
Hybrid + BGE	1.435
Redis + SBERT + BGE	**0.764**

Among all evaluated pipelines, the Redis-enhanced approach achieved the lowest average latency of **0.764 seconds**. This improvement is primarily due to semantic caching, which allows previously retrieved contexts and responses to be reused for similar queries without performing a full retrieval operation.

The SBERT + BGE pipeline achieved an average latency of **1.192 seconds**, making it the second fastest approach. Although semantic retrieval improves answer quality, embedding-based retrieval and reranking introduce additional computational overhead compared to cached retrieval.

The BM25 and Hybrid + BGE pipelines exhibited higher average latencies of **1.469 seconds** and **1.435 seconds**, respectively. The Hybrid approach combines lexical retrieval (BM25) and semantic retrieval (SBERT) before applying BGE reranking, resulting in additional processing steps that increase response time. However, this additional cost is offset by improved retrieval quality and answer grounding.

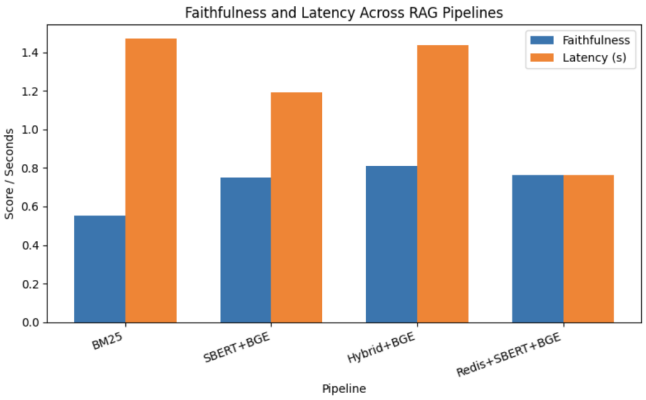

Fig. 7: Faithfulness and Latency score across RAG Pipeline.

VI. Key Observations

The trade-off between latency and faithfulness in Figure 8 illustrates that the Hybrid pipeline achieved the highest faithfulness score (**0.811**) but required a longer response time (**1.435 s**). In contrast, the Redis-enhanced pipeline maintained competitive faithfulness (**0.764**) while achieving the fastest response time (**0.764 s**). These results demonstrate that Redis caching provides substantial efficiency gains, whereas Hybrid retrieval offers the strongest answer grounding at the expense of slightly higher latency.

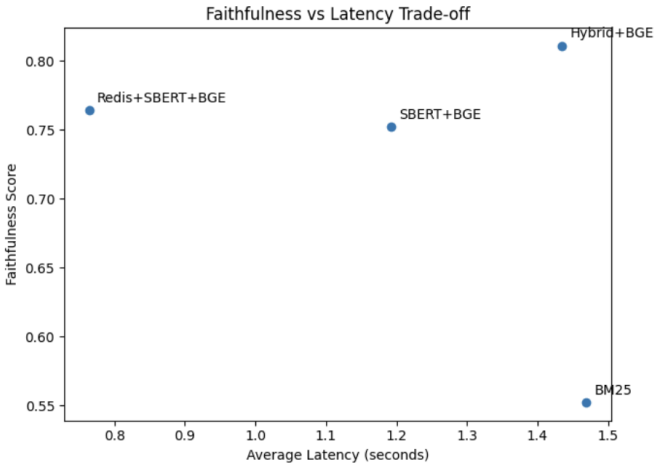

Fig. 8: Trade off between Faithfulness and Latency across RAG Pipeline.

Table IV summarizes the performance of all evaluated RAG pipelines across the three metrics used in this study: Answer Relevancy, Faithfulness, and Latency. The Hybrid Retrieval pipeline achieved the highest Answer Relevancy and Faithfulness scores.

TABLE IV: Performance Comparison of RAG Pipelines

Pipeline	Rel.	Faith.	Lat. (s)	P@5	NDCG@5
BM25	0.7394	0.5520	1.469	0.4333	0.6942
SBERT+BGE	0.7263	0.7520	1.192	**0.9867**	**0.9888**
Hybrid+BGE	**0.7516**	**0.8110**	1.435	0.9600	0.9839
Redis+SBERT+BGE	0.7408	0.7640	**0.764**	–	–

The heatmap in Figure 9 provides a consolidated comparison of all RAG pipelines across the evaluation metrics.

- Hybrid + BGE achieved the strongest overall performance, obtaining the highest Answer Relevancy (0.752) and Faithfulness (0.811) scores while maintaining excellent retrieval quality with Precision@5 (0.960) and NDCG@5 (0.984).
- SBERT + BGE produced the best retrieval results, achieving the highest Precision@5 (0.987) and NDCG@5 (0.989) scores. This indicates that semantic retrieval is highly effective at identifying and ranking relevant products.
- Redis + SBERT + BGE achieved the lowest latency (0.764 s), significantly outperforming all other pipelines in response time while maintaining competitive Faithfulness (0.764) and Answer Relevancy (0.741) scores.
- BM25 showed the weakest overall performance, with the lowest Precision@5 (0.433), NDCG@5 (0.694), and Faithfulness (0.552) scores, demonstrating the limitations of relying solely on keyword-based retrieval.
- The heatmap highlights a clear quality–efficiency trade-off: Hybrid + BGE provides the highest answer quality, whereas Redis + SBERT + BGE offers the best balance between retrieval quality and system efficiency.

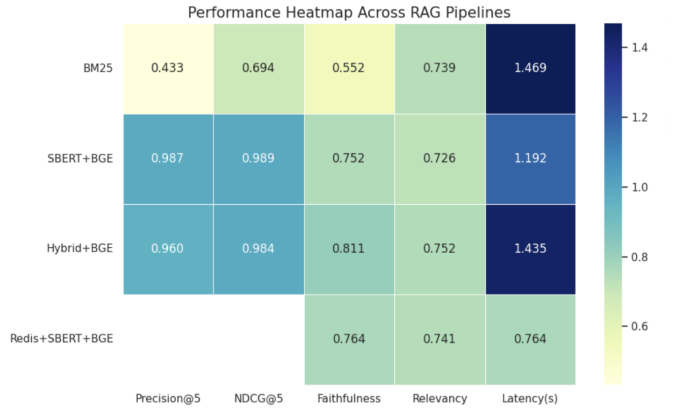

Fig. 9: Performance Heatmap across all Pipelines

Finally, the results demonstrate that semantic retrieval methods consistently outperform traditional keyword retrieval, and that combining lexical retrieval, semantic retrieval, reranking, and caching leads to the most effective RAG system.

VII. Conclusion

This study evaluated four retrieval strategies for an intelligent e-commerce search system using Answer Relevancy, Faithfulness, Latency, Precision@5, and NDCG@5 metrics. The results show that retrieval strategy significantly impacts both answer quality and system performance.

Among all evaluated pipelines, **Hybrid Retrieval (BM25 + SBERT + BGE)** achieved the best overall performance, producing the most relevant and faithful responses. The **SBERT + BGE** pipeline achieved the strongest retrieval effectiveness, while **Redis + SBERT + BGE** achieved the lowest latency through semantic caching.

Overall, the results demonstrate that combining lexical retrieval, semantic retrieval, reranking, and caching leads to a more effective and efficient RAG-based e-commerce search system. One limitation of this study is that the evaluation was performed on a relatively small set of queries and product categories. Future work may include evaluating larger datasets, exploring advanced embedding and reranking models, and incorporating additional retrieval metrics such as Context Precision and Context Recall.

References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] E. Hatcher, O. Gospodnetic, and M. McCandless, *Lucene in Action*, 2nd ed. Manning Publications, 2010.

[5] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval: The Concepts and Technology Behind Search*, 2nd ed. Addison-Wesley, 2011.

[6] Q. Ai, L. Yang, J. Guo, and W. B. Croft, "Analysis of the paragraph vector model for information retrieval," in *Proceedings of the 25th ACM International Conference on Information and Knowledge Management*, 2016, pp. 1337–1346.

[7] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *International Conference on Learning Representations (ICLR)*, 2013.

[8] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[10] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[12] J. Guo, Y. Fan, Q. Ai, and W. B. Croft, "A deep relevance matching model for ad-hoc retrieval," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2016, pp. 55–64.

[13] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, "Learning deep structured semantic models for web search using clickthrough data," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2013, pp. 2333–2338.

[14] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.